

Challenge : Optimisation des Déplacements dans un Établissement Scolaire

Laboratoire d'Algorithmes Génétiques

Contexte du Problème

Situation Réelle

Vous êtes consultant en optimisation pour une grande école qui va déménager dans un nouveau bâtiment. Le proviseur veut organiser les salles de classe pour **minimiser le temps de déplacement des élèves entre les cours**.

Challenge : Optimisation des Déplacements

Problème : Où placer les 10 salles de classe dans le nouveau bâtiment pour que les déplacements entre cours soient les plus courts possibles ?

Données fournies :

- Matrice de fréquence des déplacements entre matières
- Plan architectural du bâtiment avec emplacements disponibles
- Distances entre les différents emplacements
- Contraintes spécifiques pour certaines salles

Objectif : Trouver l'arrangement optimal des salles pour minimiser la distance totale parcourue par les élèves sur une journée type.

1 Données du Problème

1.1 Les 10 Matières à Placer

1.2 Matrice de Fréquence des Déplacements

Probabilité qu'un élève ait cours de matière i puis matière j :

Code	Matière	Spécificité
M1	Mathématiques	Nécessite tableau interactif
M2	Physique	Laboratoire équipé
M3	Chimie	Hotte aspirante
M4	Biologie	Prise de courant nombreuses
M5	Informatique	Ordinateurs fixes
M6	Histoire-Géographie	Vidéoprojecteur
M7	Langues	Système audio
M8	Arts Plastiques	Évier nécessaire
M9	Musique	Isolation phonique
M10	Éducation Physique	Accès extérieur direct

TABLE 1 – Liste des matières et leurs contraintes

$$F = \begin{bmatrix} 0.00 & 0.15 & 0.10 & 0.05 & 0.20 & 0.08 & 0.07 & 0.03 & 0.02 & 0.10 \\ 0.12 & 0.00 & 0.18 & 0.12 & 0.05 & 0.04 & 0.06 & 0.02 & 0.01 & 0.10 \\ 0.08 & 0.16 & 0.00 & 0.14 & 0.06 & 0.05 & 0.04 & 0.03 & 0.02 & 0.12 \\ 0.04 & 0.10 & 0.12 & 0.00 & 0.08 & 0.10 & 0.06 & 0.08 & 0.04 & 0.08 \\ 0.18 & 0.04 & 0.05 & 0.06 & 0.00 & 0.12 & 0.10 & 0.05 & 0.03 & 0.07 \\ 0.06 & 0.03 & 0.04 & 0.08 & 0.10 & 0.00 & 0.15 & 0.10 & 0.08 & 0.06 \\ 0.05 & 0.05 & 0.03 & 0.05 & 0.08 & 0.14 & 0.00 & 0.12 & 0.10 & 0.08 \\ 0.02 & 0.02 & 0.02 & 0.06 & 0.04 & 0.08 & 0.10 & 0.00 & 0.15 & 0.11 \\ 0.01 & 0.01 & 0.01 & 0.03 & 0.02 & 0.06 & 0.08 & 0.12 & 0.00 & 0.16 \\ 0.09 & 0.08 & 0.10 & 0.06 & 0.06 & 0.05 & 0.06 & 0.09 & 0.13 & 0.00 \end{bmatrix}$$

Plan Architectural

Interprétation : L'élément $F_{1,2} = 0.15$ signifie qu'il y a 15% de chance qu'un élève ait Maths puis Physique. Plus cette valeur est élevée, plus il est important que ces salles soient proches.

1.3 Plan Architectural du Bâtiment

1.4 Distances entre Positions

1.5 Contraintes Architecturales

- **P1, P2, P3, P4** : Côté Ouest - Pas d'accès direct à l'extérieur
- **P5, P6, P7, P8** : Côté Est - Fenêtres donnant sur cour
- **P9** : Au Nord - Proche de la cafétéria (bruit potentiel)
- **P10** : Au Sud - Accès direct à l'extérieur
- **P3, P7** : Équipées en prises électriques renforcées
- **P8, P10** : Équipées d'éviers
- **P9** : Isolation phonique renforcée

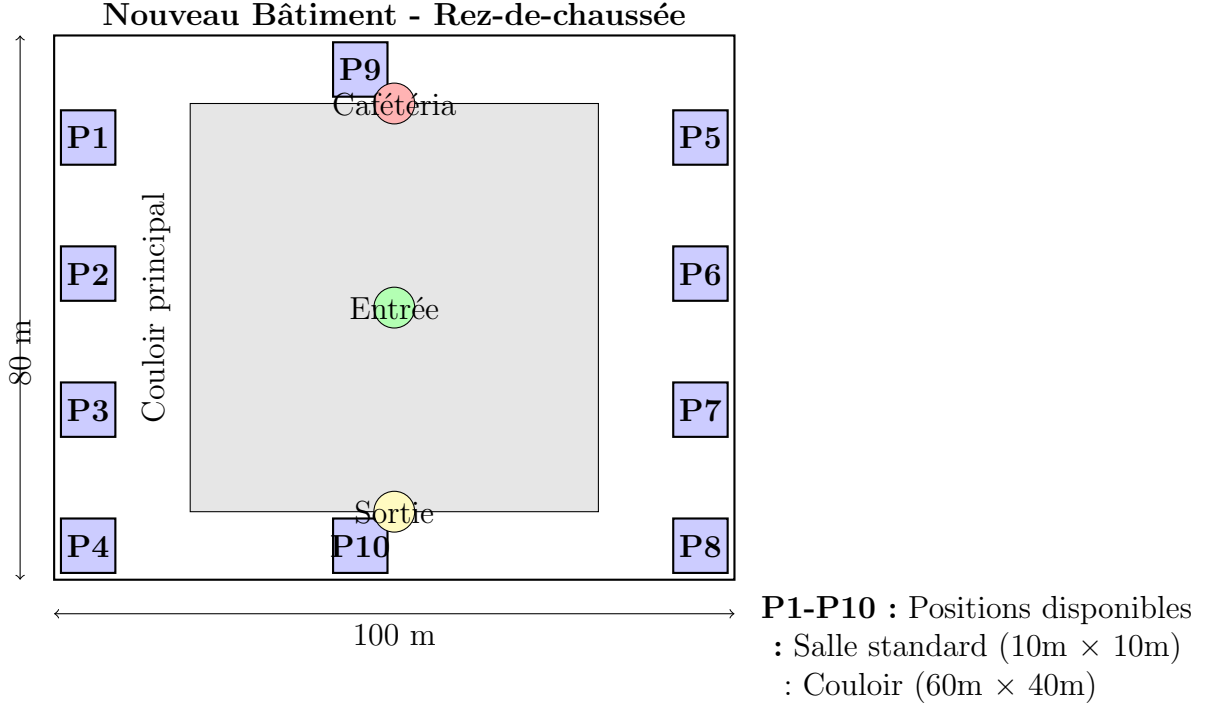


FIGURE 1 – Plan architectural avec 10 positions disponibles (P1 à P10)

2 Définition du Challenge

2.1 Objectif Principal

Challenge : Optimisation des Déplacements

Formalisation du problème :

Soit un arrangement A qui associe à chaque matière M_i une position P_j .

Fonction objectif à minimiser :

$$\text{Coût}(A) = \sum_{i=1}^{10} \sum_{j=1}^{10} F_{ij} \times D(A(M_i), A(M_j)) + \text{Pénalités}(A)$$

Où :

- F_{ij} = fréquence des déplacements de M_i vers M_j
- $D(p, q)$ = distance entre positions p et q
- $\text{Pénalités}(A)$ = pénalités pour contraintes non respectées

	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10
P1	0	20	40	60	100	85	95	110	50	65
P2	20	0	20	40	85	70	80	95	40	45
P3	40	20	0	20	95	80	70	85	55	25
P4	60	40	20	0	110	95	85	70	75	15
P5	100	85	95	110	0	20	40	60	65	95
P6	85	70	80	95	20	0	20	40	45	80
P7	95	80	70	85	40	20	0	20	55	70
P8	110	95	85	70	60	40	20	0	75	55
P9	50	40	55	75	65	45	55	75	0	70
P10	65	45	25	15	95	80	70	55	70	0

TABLE 2 – Distances en mètres entre les positions (via couloirs)

2.2 Représentation d'une Solution

Plan Architectural

Comment représenter une solution ?

Une solution est une **permutation** des 10 matières sur les 10 positions.

Exemple de chromosome : $[M3, M1, M8, M5, M2, M9, M4, M10, M6, M7]$

Cela signifie :

- Position P1 $\leftarrow$ M3 (Chimie)
- Position P2 $\leftarrow$ M1 (Maths)
- Position P3 $\leftarrow$ M8 (Arts Plastiques)
- ... etc.

2.3 Fonction d'Évaluation (Fitness)

À implémenter :

1. Calculer la distance totale pondérée :

$$\text{distance_totale} = \sum_{i=1}^{10} \sum_{j=1}^{10} F_{ij} \times D(\text{position}(M_i), \text{position}(M_j))$$

2. Ajouter les pénalités pour contraintes :

- +1000 si EPS (M10) n'est pas en P8 ou P10 (accès extérieur)
- +500 si Musique (M9) n'est pas en P9 (isolation phonique)
- +300 si Arts Plastiques (M8) n'est pas en P8 ou P10 (égout)
- +200 si Chimie (M3) n'est pas en P3 ou P7 (prise renforcée)
- +200 si Informatique (M5) n'est pas en P3 ou P7 (prise renforcée)

3. Calcul du score (à maximiser) :

$$\text{fitness} = \frac{10000}{1 + \text{distance_totale} + \text{pénalités}}$$

3 Consignes pour les Étudiants

3.1 Travail à Réaliser

Challenge : Optimisation des Déplacements

En groupe de 2-3 étudiants, vous devez :

1. **Implémenter la structure de base** : - Classe pour représenter une solution
- Fonction pour calculer la distance totale - Fonction pour calculer les pénalités -
Fonction fitness complète
2. **Implémenter l'algorithme génétique** : - Initialisation aléatoire de la population
- Sélection des parents (roulette ou tournoi) - Opérateur de croisement adapté aux
permutations - Opérateur de mutation adapté - Élitisme pour garder les meilleures
solutions
3. **Expérimenter et optimiser** : - Tester différents paramètres - Visualiser la pro-
gression - Analyser les solutions trouvées
4. **Présenter les résultats** : - Meilleure solution trouvée - Distance totale obtenue -
Temps de calcul - Analyse des paramètres

3.2 Contraintes Techniques

- **Langage** : Python 3.x
- **Interdictions** : Utilisation de bibliothèques d'optimisation toutes faites
- **Requis** : Code propre, commenté, avec tests unitaires
- **Délai** : 2 semaines

3.3 Étapes Recommandées

1. Comprendre le problème et les données
 2. Implémenter la représentation des solutions
 3. Coder la fonction fitness
 4. Créer la population initiale
-
1. Implémenter les opérateurs génétiques
 2. Faire tourner l'algorithme de base
 3. Optimiser les paramètres
 4. Analyser les résultats

4 Indices et Conseils

4.1 Indice 1 : Représentation

Plan Architectural

Pour les permutations :

- Une solution = liste de 10 éléments (les matières)
- Chaque matière apparaît exactement une fois
- L'index dans la liste = la position (index 0 → P1, index 1 → P2, etc.)

Exemple en Python :

```
# Matières disponibles
matieres = ['M1', 'M2', 'M3', 'M4', 'M5', 'M6', 'M7', 'M8', 'M9', 'M10']

# Solution example (permutation)
solution = ['M3', 'M1', 'M8', 'M5', 'M2', 'M9', 'M4', 'M10', 'M6', 'M7']

# Position de M1 dans cette solution
position_M1 = solution.index('M1') # Renvoie 1 (P2)
```

4.2 Indice 2 : Opérateurs de Croisement

Plan Architectural

Pour les permutations, le croisement simple ne marche pas !

Problème : Si on coupe et recolle, on peut avoir des doublons.

Opérateurs adaptés aux permutations :

- **Order Crossover (OX)** : Préserve l'ordre relatif
- **Partially Mapped Crossover (PMX)** : Échange des segments
- **Cycle Crossover (CX)** : Préserve les positions absolues

Recherchez ces algorithmes !

4.3 Indice 3 : Opérateurs de Mutation

Plan Architectural

Pour les permutations :

- **Swap mutation** : Échange deux éléments aléatoires
- **Inversion mutation** : Inverse un segment aléatoire
- **Insertion mutation** : Déplace un élément à une nouvelle position
- **Scramble mutation** : Mélange un segment aléatoire

Commencez simple avec swap mutation !

4.4 Indice 4 : Évaluation de Performance

Plan Architectural

Métriques à suivre :

- Meilleur fitness par génération
- Fitness moyen par génération
- Temps de convergence
- Diversité de la population

Objectifs de performance :

- Fitness ≥ 50 : Bon
- Fitness ≥ 80 : Excellent
- Fitness ≥ 90 : Optimal probable

5 Questions de Réflexion

5.1 Questions pour l'Analyse

1. Quelle est la complexité de votre fonction fitness ?
2. Comment évaluez-vous la diversité de votre population ?
3. Quel opérateur de croisement donne les meilleurs résultats ?
4. Comment le taux de mutation affecte-t-il la convergence ?
5. Avez-vous des solutions équivalentes avec le même fitness ?

5.2 Extensions Possibles

- **Multi-objectif** : Minimiser aussi le nombre d'élèves dans les escaliers
- **Dynamique** : Tenir compte des horaires différents
- **Réaliste** : Ajouter la capacité des salles
- **Interactive** : Interface pour visualiser les solutions

6 Critères d'Évaluation

6.1 Grille d'Évaluation

6.2 Attendus Minimum

- Algorithme génétique fonctionnel
- Fitness ≥ 30 atteint
- Code exécutable sans erreurs
- Rapport avec résultats de base

Critère	Insuffisant	Satisfaisant	Excellent
Implémentation AG	Code incomplet	AG fonctionnel	AG optimisé
Fitness atteint	≤ 30	30-70	≥ 70
Qualité du code	Peu structuré	Bien structuré	Professionnel
Expérimentations	Aucune	Quelques tests	Analyse complète
Rapport	Minimal	Complet	Insightful

6.3 Bonus

- Interface de visualisation
- Comparaison avec d'autres algorithmes
- Analyse statistique des résultats
- Optimisation des paramètres automatisée

Objectif pédagogique : Comprendre comment formuler un problème réel en problème d'optimisation et appliquer un algorithme génétique pour le résoudre.